

Programmer commands

The 42 development verbs a standard account does not get, and what withholding them does and does not do.

A **standard** account gets 82 verbs — enough to run an application and nothing that edits code or data in bulk. A **programmer** account gets those plus the 42 below. An **administrator** account gets all of them, plus 23 more.

This split does not exist in OpenQM, where every account gets the same VOC. None of the verbs below is new; what is new is that an account has to be created as `programmer` or `administrator` to receive them.

```
create.account user fred programmer both
```

What is withheld, by function

COMPILE, CATALOGUE AND RUN

<code>basic</code>	compile SD BASIC source
<code>catalog</code> · <code>catalogue</code>	add to the catalogue
<code>delete.catalog</code> · <code>delete.catalogue</code>	remove from it
<code>compile.dict</code>	compile dictionary items
<code>run</code>	run a compiled program
<code>map</code>	show a program's map
<code>generate</code>	generate source
<code>phantom</code>	start a background process

Cataloguing globally requires an administrator. Adding to or removing from the system-wide catalogue is an administrator action; private and local cataloguing still work in a programmer's own account. That is a separate control from the VOC tier.

EDIT AND DEBUG

<code>ed</code>	the line editor. Needs nothing installed
<code>edit</code>	a full-screen editor — opens the record in Microsoft Edit
<code>micro</code>	a full-screen editor — opens the record in micro
<code>debug</code>	the BASIC debugger
<code>pstat</code> · <code>pdebug</code> · <code>pdump</code> · <code>dump</code>	process introspection

The debug family moved from administrator to programmer deliberately — a programmer needs these to debug their own code.

EDITORS

There are two, and they behave identically. The verb chooses the editor and nothing else changes:

<code>edit</code>	Microsoft Edit — ships in current Windows builds
<code>micro</code>	micro — never ships with Windows; the installer fetches it

```
edit bp myprog
micro bp myprog
edit dict customers name
```

`edit` **used to be a second name for `ed`, and is not any more.** If you have been typing it to get the line editor, you will now get a full screen.

Either verb writes the record to a working copy, opens the editor on it, reads it back, and asks whether to save. For a `bp` record it then offers the compile and the catalogue, which is the loop the old `micro` verb had.

Both are terminal editors, so both work over ssh as well as at the console. Where a machine does not have one, the SD installer installs it; if that could not be done, the verb says so and names the command that installs it.

`C:\ProgramData\SD\install-editors.log` records what the installer found.

Only `micro` highlights SD basic. Microsoft Edit has no syntax highlighting at all, which is the one real difference between the two verbs:

<code>micro</code>	statements, reserved words, intrinsic functions, <code>@variables</code> , <code>\$directives</code> , labels, strings, numbers and comments
--------------------	--

edit plain text

It applies to a bp record and to nothing else. SD names the working copy so that micro can recognise the language — a record edited out of any other file is treated as plain text, which is correct for a VOC entry or a data record.

The word lists are generated from the compiler. They come out of BCOMP's own tables — **218 statements, 37 reserved words and 176 intrinsic functions** — so the highlighting cannot drift from the language. **If a name you expect is not coloured, that is worth reporting:** it means the two have come apart, which is exactly what generating them was meant to prevent.

Nothing is installed into your profile. SD ships the rules with itself and points micro at them, so they work for every account on the machine including the ones that cannot log in to Windows.

ed **is unaffected and is still there.** Nothing has been taken away.

WHAT THE EDITORS ARE GOOD FOR, AND WHAT THEY ARE NOT

They are text editors, so they suit a record whose content is lines of text:

BASIC source in a bp file	what they are for
VOC records	fine — a VOC record is a few short fields
Dictionary records	fine for a simple one; see the limit below
Data records with multivalues	fine — see the tokens below
Data records with subvalues	fine — see the tokens below

A field is a line and that part needs no explanation. SD writes the working copy with one field per line, so moving between fields is moving between lines.

A value mark is not a line, and neither is a subvalue mark. Both are control characters an editor cannot show, so each has a token you can type:

TYPE	TO GET
~~	a value mark
~`	a subvalue mark

SD converts marks to tokens on the way into the editor and tokens back to marks on the way out, so multivalues and subvalues are both ordinary text while you are editing.

SMITH~~JONES~~BROWN

is a three-value field, and

RED~`BLUE~~GREEN

is two values, the first of which has two subvalues.

A record that cannot be written this way is refused, not mangled. Some records would come back different from how they went in — one that already contains `~~` as data, for instance, or one with a `~` sitting immediately before a mark, where the tilde and the token run together. Before opening the editor, SD converts the record and converts it back; **if the result is not what it started with, the verb refuses and names** `ed`, which needs none of this.

Text marks are not converted, and are covered by the same refusal rather than being left to surprise you.

A compiled dictionary record is truncated to its first 15 fields while you edit it, and recompiled with `cd` when you save — the same thing the old `micro` verb did.

GIVE THESE VERBS ONLY TO PEOPLE YOU TRUST

An editor can write anywhere its user can write. It opens the record you named, but nothing stops the person then opening any other file on the machine that their Windows account may open — inside the SD data tree or outside it altogether. **That is not a hole in SD; it is what an editor is**, and it is the reason these verbs are behind `OS.EXECUTE` permission and not merely behind the programmer tier.

So `os.users` **field 2 is a statement of trust in a person, not a convenience.** Before granting it, ask the same question you would ask before giving somebody the shell — because in terms of what they can reach on disk, you are.

Neither editor can run a command, so neither is a shell. **What they are is read and write access to the filesystem, with the account's own Windows permissions.** See Security.

BOTH EDITORS NEED `OS.EXECUTE` PERMISSION AS WELL AS THE VERB

Having the verb is not enough. An editor runs outside SD, so reaching one is reaching the operating system — and who may do that is **field 2 of your record in** `os.users`, the same field that governs `OS.EXECUTE` from inside a program. Two gates, and both have to pass:

the VOC tier	decides who has the verb — programmer and administrator
--------------	--

<code>os.users</code> field 2	decides whether it runs
-------------------------------	--------------------------------

An elevated session passes on its own, exactly as `sh` does, so an empty list cannot lock the machine's own administrator out.

A missing record, or a missing file, means no — the same direction `sh` fails in.

If you have the verb and not the permission you get told so by name, and told what to ask for:

```
edit is not available to fred.  
It runs an editor outside SD, so it needs OS.EXECUTE permission: field 2  
of your record in the SD system file os.users, which only an administrator  
can change.  
ed, the line editor, needs none of this.
```

An administrator grants it — see Administrator commands.

A session with no terminal is refused first and separately: an API session or a piped script has nowhere to draw a full screen, and is told that rather than being told about `os.users`.

WHAT AN EDITOR CAN REACH, and it is worth knowing before you grant it. An editor can open any file the person running it is allowed to open, so both verbs reach beyond SD's own files. Neither is a shell — neither editor can run a command. That is what field 2 is deciding, and it is why the verb alone was not made enough. See Security.

OVER SSH

A terminal editor is the point of an ssh session. An ssh session reaches SD through a terminal like any other, and SD hands the editor that terminal rather than reading it through a pipe.

AN ssh SESSION CAN NEVER BE ELEVATED, though, so an administrator arriving that way needs an `os.users` entry the same as anybody else — being an administrator is not enough on its own over ssh.

If an editor misbehaves over ssh and not at the console, that is worth reporting with the terminal you connected from.

The removed full-screen editors are a different matter: `sed`, `update.record` and `modify` are gone and are not coming back. See Not in SD Core.

FILES

<code>create.file</code> · <code>delete.file</code> · <code>clear.file</code>	the life of a file
<code>configure.file</code>	change a file's configuration
<code>analyse.file</code> · <code>analyze.file</code>	report on a file's internals
<code>fstat</code>	file statistics
<code>hsm</code>	hashed-file statistics monitoring
<code>set.trigger</code>	attach a trigger

<code>cd</code>	change directory
-----------------	------------------

INDEXES

`create.index` · `delete.index` · `build.index` · `make.index` · `list.index`

BULK RECORD EDITING

<code>copy</code> · <code>copyp</code>	copy records
<code>delete</code>	delete records
<code>rename</code>	rename records
<code>reformat</code> · <code>sreformat</code>	reformat
<code>sort.item</code>	sort
<code>cname</code>	change a record's name
<code>delete.common</code>	clear a common block

What the split does and does not do

It is a posture, not a boundary. An administrator can copy any verb into any account's VOC afterwards. A reduced VOC is where an account **starts**, not something SD enforces at run time.

What actually is enforced is elsewhere, and worth knowing before you rely on the tier for anything:

File permissions	Windows ACLs on the data tree — see Security
Where an account may sign in	the <code>sdsshonly</code> deny rights — see ssh access
Reaching the operating system	the <code>os.users</code> permit list, both <code>sh</code> and <code>OS.EXECUTE</code>
What an API session may open	the containment gate, rooted at the account the session stands in

A programmer with `OS.EX` set has the operating system, whatever their VOC says, because they can write and run BASIC. That is why field 2 of `os.users` exists and why it is now enforced — see Administrator commands.

Two things to know when you compile

`basic` no longer creates an object file it can never open again. Compiling into a reused file name previously produced an object SD could not subsequently open.

Object code and the catalogue are replaced on upgrade. The compiled programs, the BASIC source SD ships, the messages, include records and VOC templates are all overwritten by a new release. **Anything you have written into the SDSYS `bp` file, and anything you have compiled from it, survives an upgrade untouched** — SD now ships nothing into that file at all, so it is created empty and is yours.

The full list of the 42

`basic` · `catalog` · `catalogue` · `delete.catalog` · `delete.catalogue` · `compile.dict` · `cd` · `generate` · `phantom` ·
`run` · `map` · `debug` · `ed` · `edit` · `micro` · `create.file` · `delete.file` · `clear.file` · `configure.file` ·
`analyse.file` · `analyze.file` · `fstat` · `hsm` · `set.trigger` · `create.index` · `delete.index` · `build.index` ·
`make.index` · `list.index` · `copy` · `copyp` · `delete` · `rename` · `reformat` · `sreformat` · `sort.item` · `delete.common` ·
`cname` · `pstat` · `pdebug` · `pdump` · `dump`

This list was 42, then 41 when `modify` was removed from SD Core, and is 42 again now `micro` is back. A standard account's count did not move on either occasion: both verbs were on both sides of the arithmetic at once — in `NEWVOC` and in the standard tier's omit list.